

Programming by Non-programmers

LANCE A. MILLER*

*IBM Thomas J. Watson Research Center, Yorktown Heights,
New York 10598, U.S.A.*

(Received 5 June 1973)

Non-programmers were asked to organize natural English commands of a laboratory programming language into programs for solving name-sorting problems. The problems differed in the sort concept to be programmed (conjunction vs. disjunction) and in the form of expression of the letter tests to be made on the names (affirmation vs. negation.)

Programming performance was found to be impaired with disjunctive concepts and with letter tests involving negation. Different classes of program structure were identified and were associated with certain problem conditions and error measures. An influence of prior experience with procedures on performance was suggested. Program debugging and testing performance was characterized.

Introduction

The services of programmers, in creating and maintaining program software for computer applications, are becoming the key factor in the economic success and growth in the computer industry. This is due, in part, to the falling costs of computer technology on the one hand, and the rising costs of programming services (and program errors) on the other (cf. Sackman, 1968; Brophy, 1970; Hewitt, 1972). Consequently, there is considerable interest in improving the productivity and quality of programming.

Unfortunately, very little is known about the behavioral processes and problems involved in programming. Such work as does exist is most often oriented towards the evaluation of alternative computer systems (e.g. time-sharing vs. batch systems) where differences among programmers are found to be much greater than the small differences found between systems (e.g. Sackman, 1970). In the programmer selection and training area very little progress has been made in identifying traits which predict programming

*Requests for reprints to Lance A. Miller, IBM Thomas J. Watson Research Center, P.O. Box 218, Yorktown Heights, New York 10598, U.S.A.

success (e.g. Mayer & Stalnaker, 1967). The provocative book *The Psychology of Computer Programming* (Weinberg, 1971) is perhaps most telling of the state of the art, in view of the paucity of experimental data it contains. (For an annotated partial bibliography of the behavioral literature on programming, cf. Gould, Doherty, & Boies, 1971).

There are good practical reasons, then, for studying the process of computer programming. In addition, perhaps of equal importance, the investigation of programming can serve as a vehicle for fundamental problem-solving research. Three points may be emphasized in this regard. First, programming, as an instance of problem-solving behavior is not—it seems—easily subsumed under the usual classifications of problem-solving activities (e.g. Miller, 1968; Maier, 1970). Rather than involving the learning of a procedure by subjects (as in concept learning; e.g. Bourne, 1966) or the inference by the experimenter of procedures followed by subjects in the solution of problems (e.g. Duncker, 1945), programming requires the objective specification of a sequence of actions necessary to accomplish some objective—what may be called *procedure specification*.^{*} A critical characteristic of this activity is that it is not sufficient that subjects be able to provide a solution for a particular problem; it is also necessary that the solution procedure be specified in such a way that it may be successfully followed by other, designated, agents. Investigation of programming behavior, then, provides an opportunity for extending the domain of problem-solving research into the heretofore relatively unexplored area of procedure specification.

A second point concerning programming as problem solving is that the complete programming process, from initial problem specification to final program production and verification, would appear to include most of the activities presently classified under problem solving. Creativity, planning, and inductive reasoning appear to be involved in problem formulation and generation of a solution algorithm; deductive reasoning would seem to be required in the specification of successive actions, particularly when the actions to be taken are contingent upon the state of external variables; certainly, diagnostic capability is necessary in the detection and correction of errors in the procedure. Thus, programming offers a microcosm for the investigation of many aspects of problem-solving behavior within a unified domain of activity.

^{*}A proposed general definition of procedure specification is: "Objective specification of a sequence of actions to be accomplished by designated agents so as to realize an objective defined in terms of certain classes of consequent events (outputs) resulting from transformations applied to certain classes of antecedent events (inputs), which action sequence may be dependent on both the nature of the inputs and the results of transformation actions."

Third, the programming environment offers several important advantages for control of information, data collection, and performance evaluation, in contrast to other task situations. Performance observation and data capture are facilitated by the ready availability of automatic computer-programmer transaction-recording techniques (e.g. Boies & Gould, 1971). Exceptional control over the availability of alternative methods of accomplishing the task and accompanying information sources is made possible by controlling the characteristics of the programming language and the language-checking facilities. Finally, the behavior of programs in computers is measurable, and there exist some standards and procedures for evaluating programs in terms of performance efficiency, sufficiency, and "quality".

An Experimental Study of Programming

The study reported here is the first of a series intended to provide information useful not only for the practical problem of improving programming performance but also for extending the basic understanding of problem-solving behavior. One important characteristic of this experiment is the use of subjects who had no prior experience in programming (hereafter called "naive"). It is presumed *a priori* that programming requires general cognitive abilities which are in the repertoire of most normal adults. Use of naive subjects over experienced programmers may permit detection of the characteristics of these processes unconfounded by differential training, experience, expectations, and attitudes associated with the professionals.

A second aspect of the study is the creation and use of a laboratory programming language. Existing programming languages were judged undesirable for use in this initial exploratory study of programming with naive subjects (cf. Methods section).

Both independent variables manipulated in the study were concerned with the problems given subjects. All problems required that two particular letter positions in names on cards be examined for certain "target" values. One problem variable was the manner in which the target value was expressed in the problem—either affirmatively (e.g. checking for the second letter being "L") or negatively (e.g. checking for the second letter *not* being "L").

The second problem variable concerned the logical relation between outcomes of letter tests that had to be satisfied before certain actions could be taken. In one case, that of conjunction, both targets had to be detected. In the second case, that of inclusive disjunction, only one or the other targets needed to be detected.

Manipulation of these variables provided one means of providing a range of problem difficulty for which programming performance could be sampled. On an *a priori* basis differential effects on performance of these variables could be interpreted in two ways. First, evidence from other areas suggests that subjects may experience conceptual difficulties with certain levels of these variables. In concept-learning research, disjunctive concepts have proven more difficult to attain than conjunctive ones (e.g. Bourne, 1966), and in tasks involving the understanding of logical assertions, negatively-expressed concepts have been found to be more difficult than those affirmatively expressed (e.g. Wason, 1959). Thus, these levels of the problem variables might also be expected to provide greater difficulty in the quite different task of programming, on the basis of generalizeable differences in conceptual abilities.

Alternatively, the problem variables may differentially affect performance by complication of the task-specific process of translating these logical characteristics into a sequence of actions and tests in the programmed solution. In this view it is not any differences in general conceptual capability to understand or infer these logical characteristics which would produce performance differences, but the differential difficulty of translating these concepts, expressed generally in the problem, into a sequence of programming commands.

Experimental Method

SUBJECTS

The subjects were 12 undergraduate students from two local colleges who were paid for their participation in the experiment.

DESIGN

Each subject solved six problems, where the problem order was determined by a (replicated) 6×6 balanced Latin square. The problems were combinations of two variables having two and three levels, respectively: logical relation ("and" vs. "or") and form of test expression (both variables expressed affirmatively, both negatively, and mixed). The subjects were randomly assigned to rows within squares in order of their appearance for the experiment.

PROBLEMS

The problem scenario was that of a filing clerk or secretary who received in an "input box" cards on which were printed last names and which were

to be sorted into boxes on the basis of two of the letters in the names. For example, one problem ("or", mixed expression) was:

"Put a card in box 3 if either the name's second letter is *not* 'L' or if its last letter is 'N', or both—only one condition must be satisfied. Count the number of cards in box 3 using counter 1. Put the remaining cards in box 2."

The remaining five problems preserved the exact form and wording of the above, with the exception of substituting different numbers and different logical operators.

The subjects were required to use a set of laboratory programming commands (described below) in solving these six problems. Each of the problems required a minimum of 13 of these commands: three to initialize ("empty") the boxes and counters used, three to test for the presence of remaining cards in the input box or to examine the specified letter positions for the target letters, three for moving cards from or to boxes, one for counter incrementing, two to transfer control unconditionally to previous commands ("go to" commands), and one command to indicate the end of the program.

The problems were therefore almost identical with respect to wording and identical with respect to the number and type of commands. The important differences were in the specification (by giving address labels) of the commands to be executed given the detection or non-detection of the target letter values. While these differences produce quite different processing of name cards, the differences in the transfer-of-control structures for "and" vs. "or" problems is rather minimal, particularly compared to the differences in control structures which exist in real programs.

EXPERIMENTAL PROGRAMMING LANGUAGE

The 19 programming commands of the laboratory programming language (including transfer addresses and address labels) are shown in Fig. 1 as they were represented to subjects. Different card transfer and letter test commands were required by the problems to help prevent stereotyped responding.

Commands M–O provide for resetting data variables from values which would have resulted from their use in previous problems. Commands A–E are conditional tests on the state of external variables, having two possible outcomes, "yes" and "no". In these commands the addresses of other commands (B1–B7, or NX indicating transfer to the next command) had to be specified to indicate where control was to be transferred, depending on the outcome. Commands G–I and L are commands indicating the manner in

which cards were to be moved, and commands J and K produce increments in the counters. Command R is an unconditional transfer-of-control command requiring the specification of some address pointer indicating the (out of sequence) command to be executed next. Finally, command S indicates that processing of data should be terminated.

COMMAND SET

(A) IS FIRST LETTER "A"?	YES:	NO:
(B) IS THIRD LETTER "D"?	YES:	NO:
(C) IS FOURTH LETTER "R"?	YES:	NO:
(D) IS SECOND LETTER "L"?	YES:	NO:
(E) IS LAST LETTER "N"?	YES:	NO:
(F) ANY CARD IN INPUT BOX?	YES:	NO:
(G) PUT CARD IN BOX 1		
(H) PUT CARD IN BOX 3		
(I) PUT CARD IN BOX 2		
(J) ADD ONE TO COUNTER 2		
(K) ADD ONE TO COUNTER 1		
(L) TAKE NEXT INPUT CARD AND LOOK AT NAME		
(M) EMPTY BOX 2		
(N) EMPTY BOX 1		
(O) EMPTY BOX 3		
(P) EMPTY COUNTER 1		
(Q) EMPTY COUNTER 2		
(R) GO TO —		
(S) STOP		

POSSIBLE ADDRESSES OF COMMANDS

B1 B2 B3 B4 B5 B6 B7

POSSIBLE POINTERS

B1 B2 B3 B4 B5 B6 B7 NX (FOR "NEXT")

Fig. 1. Programming commands available for selection (by letter).

This command set was sufficient for the expression of program solutions to the restricted problems of this experiment. Use of these commands, already simplified in view of their expression as complete English sentences (missing, at most, one or two articles of speech), was further simplified by permitting subjects to select commands by typing in only the identifying letter, rather than the complete command. In contrast to other programming

situations, the present requires only command recognition and selection as opposed to the generation of commands. Existing "real" programming languages were judged to be too difficult and too demanding to acquire and employ in a reasonable amount of experimental training time, particularly since subjects had no previous exposure even to the concept of specifying procedures in any language. A further drawback of most existing languages is that they typically provide many opportunities for "syntactic" errors—specifications of command which violate one or more of the linguistic conventions or lexicon restrictions of the programming language. With the present programming language, no syntactic errors were possible (specification of unallowable addresses or pointers was detected immediately and prevented). Thus, the only errors that programs could contain were those of a "conceptual" nature—leading to incorrect processing of data rather than simply incorrect grammatical constructions. Finally, in order to permit modification of programs before completion and to provide for prompting for addresses and pointers (as well as checking for those now allowed) an interpreted programming language (such as the present APL-based language) was required; most existing widely-used programming languages would not provide such features (cf. Sammett, 1969, for a discussion of programming languages).

A correct program using these commands is shown in Fig. 2 for the "or" mixed-problem example given previously.

- (1) EMPTY BOX 3
- (2) EMPTY BOX 2
- (3) EMPTY COUNTER 1
- (4) B1: ANY CARD IN INPUT BOX? YES: NX NO: B6
- (5) TAKE NEXT INPUT CARD AND LOOK AT NAME
- (6) IS SECOND LETTER "L"? YES: NX NO: B3
- (7) IS LAST LETTER "N"? YES: NX NO: B4
- (8) B3: PUT CARD IN BOX 3
- (9) ADD ONE TO COUNTER 1
- (10) GO TO B1
- (11) B4: PUT CARD IN BOX 2
- (12) GO TO B1
- (13) B6: STOP

FIG. 2. Correct program sequence for a disjunctive, mixed-expression problem (second letter not-"L" or last letter "N").

APPARATUS

A typewriter terminal (IBM 2741 Selectric) was employed as the input-output device for the experiment. This terminal was connected via telephone lines to an IBM 360/91 computer supporting an interactive time-sharing programming system. This system was reliable (available 95% of the time when experiments were being run) and fast (about 90% of the time less than 1 sec intervened between subjects' response and some system output).

The program controlling the experiment was written in the APL programming language (cf. Pakin, 1968). Besides interpreting subjects' requests for command selection, prompting and checking addresses and pointers, the program permitted display and editing of commands at any point.

Each time control was passed to the subject the latency of the next type-written response was measured and coded specifically as to the nature of the activity resulting. The system provided for measurement of response latency ("open keyboard time") to 1/60 of a second. The potential error of measurement due to polling delays in the system was estimated as being in the range of 2-80 msec.

TRAINING

The first phase of instruction, delivered via a recorded tape and headphones, employed an example set of 2×4 in cards on which were printed commands. These cards were placed as a sample program on a long tilted stand which was divided into segments numbered from left to right. Behind the stand were various small boxes labeled "input box", "box 1", etc., as well as shallow dishes with tokens, representing the counters. Conceptually, the card stand served as a physical analogue of the control structure of the program, while the boxes represented the data structure. Each subject was individually stepped through the program by means of the recorded instructions, with particular attention given to the conditional transfer-of-control commands (commands A-E). At one point subjects used one hand to point at a command to be executed and performed the designated activity with the data boxes using the other hand.

In the second phase of the instructions, subjects were stepped through the generation (as contrasted to the execution) of a program. A card-sorting problem involving only a single letter test (as in the program of the first phase) was described, and then the instructions asked subjects to select the commands from a large pile of commands cards. The subjects wrote pointers on the cards and affixed address label tags to card edges. Following the supervised program generation, subjects were given a second problem in which they organized the command card without help.

Finally, in the third phase, the experimenter supervised subjects during solution of a one-test problem at the typewriter terminal, illustrating the command-selection procedure and the editing/display capabilities. Assistance was provided only when subjects were constructing clearly incorrect command sequences and only after several minutes following an error (to permit subjects to detect their own errors).

The complete training program required about 55 min for completion and was quite successful in conveying the concepts of procedure specification to subjects as well as instructing them in the use of the particular programming language.

EXPERIMENTAL PROCEDURE

Following training, the experimenter invoked the experimental APL control program and then left the subject at the terminal to complete the six programming problems. Upon finishing a problem, subjects were permitted to take a break if desired. Sequencing of problems was under program control. Completion of the experiment, including all breaks, unscheduled computer failures (about one per subject), and miscellaneous non-programming time, required from 1–4 days, for 2–5 hr per day.

The procedure within a problem was as follows. A problem was typed out, which subjects read, and then they began the problem by depressing the carriage return key. At this point the text “next command (mod, dis, or fin)” was typed out by the computer. This prompt indicated that subjects could enter the letter of the next command they desired (cf. Fig. 1) or else choose one of several options: “mod”, to modify their program (inserting, deleting, or changing commands); “dis”, for displaying their program up to that point; or “fin”, to indicate that they had finished completion of their program and were ready for it to be tested.

Given that subjects entered a command letter, the next prompt asked if they wished the command to have an address (which they entered, or typed “no”). For conditional transfer-of-control commands (commands A–E; cf. Fig. 1), additional prompts were given asking for address pointers for the “yes” and then the “no” outcomes. For the “go to” command, a prompt requesting the single address-pointer was given. Following completion of all information for a command, the complete command was typed out on the right side of the paper (left side being reserved for interactions). The display option typed out all previously prepared commands. When subjects typed “mod” the numbered list of alternative editing possibilities was typed out; Subjects then chose the desired alternative by typing in the appropriate number. The number of the command to be modified (or proximal to

an insertion) was then requested, followed by a request for any new information.

When subjects indicated that they had completed program organization (by entering "fin"), the address of commands and the pointers in conditional test statements were compared to determine if there were any inconsistencies. If so, control was returned to the subject with the instruction to correct the problem. Programs containing no address-label problems were submitted to a simulation program which performed the operations prescribed by the subject's program on a specially-prepared set of input name-card data to determine if the program completely fulfilled the problem requirements. If any error was detected, the subject was instructed to locate and correct the error, being returned to the editing functions. The data used to test the program were printed out, and the result of execution of the subject's program on this data set was indicated. For testing purposes, all data variables had an initial pre-problem assignment. Failure to initialize variables used in the program would result in these prior values being present after execution—indicating initialization errors. Eight test names were used to test the programs. Infinite loop errors were detected by program statements which counted the number of the subject's commands executed and stopped execution when five times the executions required by a correct program had occurred. The subjects who had incorrect programs continued modifying their programs until no errors were detected.

Before advancing to the next problem, the last 10 subjects run were asked to select from the test data of eight names a set which they considered would be both minimum and sufficient to test their program for correctness.

Results and Discussion

Initial program-organization performance was analyzed in terms of six primary measures: total elapsed time, total response latency (cumulation of time when the computer system was waiting for a typed response), preparation time (latency following problem presentation and before selection of the first command), total number of commands, number of program modifications, and number of program displays. In addition, there were 20 specific response-latency measures: four for each of five classes of commands, which classes were initialization, counter/card movement, conditional, unconditional, and termination (cf. Fig. 1). The total response latency, measured from a command prompt after which one of these classes was selected to the prompt for the next command, was one of the four measures for each class. The other three measures were partitions of this total

time: command choice time (measured from prompt to command-letter entry), edit choice time (time to choose a modification alternative prior to choosing a particular class of command), and address choice time (time to select an address for a command or enter "box").

INITIAL PROGRAM PREPARATION

Several of the performance measures indicated significantly reliable effects of the problem variable of logical relation ("and" vs. "or") as shown in Table 1.

TABLE 1

Problem condition means for performance measures showing significant effects ($P < 0.05$) of the problem variable of logical relation

Performance measure	Problem conditions	
	"And"	"Or"
Total time	16.6 (min)	22.4 (min)
Choice time for conditional commands	13.61 (sec)	26.93 (sec)
Edit choice time (conditional commands)	0.75 (sec)	8.91 (sec)
Choice time for functional commands	18.10 (sec)	45.54 (sec)
Total number of program modifications	0.97	2.47
Total number of program displays	1.33	2.39

According to the usual interpretation of relative reaction times or latencies, the first four measures indicate that "or" problems were more difficult than "and" problems. Not only were about 6 min more of decision time required for "or" problems, but a substantially greater amount of time was required to select conditional and counter/card movement commands and to indicate the decision to modify the existing program before entering a conditional command. The finding of significant effects only for the conditional and counter card movement command-type measures is reasonable since correct programs for both "and" and "or" problems are identical for the three other classes of commands.

The occurrence of more than twice as many program modifications for "or" problems, and the correlated greater number of displays, indicates that subjects initial plan of organization had to be modified with some frequency for "or" problems, but very seldom for "and" ones. Greater difficulty for "or" problems is suggested, in particular, by the 12-fold greater time required to choose a program-modification alternative prior to choosing a

conditional command. These findings, together, suggest not only that subjects were uncertain as to the correct sequential specification for "or" problems but also that given a judgment that the present organization was incorrect, subjects were uncertain as to the corrective action to be made.

In subsequent analysis, the total latencies for each of the five command types were summed; the proportion of the total latency for each command type was then computed for both types of problems. The proportion for completing the termination command for "or" problems was about twice that for "and" problems (0.22 vs. 0.09) despite the fact that this command does not differentiate between correct programs for the two kinds of problems. The "stop" command was the last command entered in almost all cases, with the testing of the program to follow. With this in mind, it seems reasonable to interpret the proportionally longer time to enter the termination command for "or" problems as reflective of subjects' greater uncertainty of the "correctness" of such programs.

A second finding of the above analysis was that in "and" problems subjects spent proportionately about half as much time in choosing and completing counter/card movement commands as with conditional commands (0.18 vs. 0.37) while in "or" problems the proportions were about equal (0.27 vs. 0.29). These differences indicate different allocations of decision or "attention" time for the two command classes in the two different kinds of problems.

Almost the same amount of total time was required for choosing and completing conditional commands in both kinds of problems (6.14 vs. 6.50 min). Since a conditional command of necessity preceded counter/card movement commands, it appears that the specification of conditional commands facilitated specification of subsequent commands for "and" problems but not for "or" ones. An implication is that the organizational planning involved with "or" problems was much more local and less extensive than with "and" problems. If so, the greater program-modification activity with "or" problems can be attributed to inadequacies presumably inherent in partial, locally limited, organizational planning.

No differential effects of the second problem variable form of test expression were detected in the above analyses. However, other lines of analysis, given below, do indicate an effect of this variable.

INCORRECT PROGRAMS

After a subject had completed organizing his program it was executed for a test sample of names by the controlling APL program. The result of execution was compared against the results which should have been obtained for the

particular problem. If there were any discrepancies, the subject had to revise his program.

Nine of the 12 subjects had at least one incorrect program out of six (one subject had incorrect programs for four problems; five had incorrect programs for two problems; three had incorrect programs for only one problem). For the 17 problems in which incorrect programs were prepared, there were a total of 32 rejections by the simulator, there being a range of from one to six rejections per problem. The subject who had incorrect programs for four problems accounted for 13 of these rejections (for his mixed "or" and both-negative "or" problems, there were six and four rejections, respectively). The modal number of rejections for the 17 incorrect-program problems was 1 (11 out of 17 instances).

TABLE 2

Frequency distribution in the six problem conditions of (a) instances of one or more incorrect programs; (b) number of incorrect programs submitted; (c) number of errors in incorrect programs involving conditional commands; (d) number of errors in incorrect programs involving non-conditional commands

Logical relation	Form of text expression			Row totals
	Both affirmative	Mixed	Both negative	
"And"	(a) 1	(a) 4	(a) 0	(a) 5
	(b) 2	(b) 4	(b) 0	(b) 6
	(c) 0	(c) 3	(c) 0	(c) 3
	(d) 2	(d) 2	(d) 0	(d) 4
"Or"	(a) 2	(a) 5	(a) 5	(a) 12
	(b) 3	(b) 13	(b) 10	(b) 26
	(c) 2	(c) 16	(c) 12	(c) 30
	(d) 2	(d) 10	(d) 0	(d) 12
Column totals	(a) 3	(a) 9	(a) 5	(a) 17
	(b) 5	(b) 17	(b) 10	(b) 32
	(c) 2	(c) 19	(c) 12	(c) 33
	(d) 4	(d) 12	(d) 0	(d) 16

The distribution in the six problem conditions of the number of problems for which (1 or more) incorrect programs were submitted is shown as entry (a) in Table 2. Entry (b) shows the distribution of the 32 incorrect programs.

The marginal row totals of these two measures replicate the greater difficulty of "or" problems previously found with latency measures. The column totals suggest an effect of the second variable, the forms of test expression, where problems involving at least one negatively-expressed test are more difficult than problems expressing both tests affirmatively; in particular, the mixed condition appeared to be the most difficult. The usual frequency statistics are inappropriately applied to these data of Table 2 in view of the fact that individual subjects contributed differently to the data, and the independence requirements of such tests are violated.*

Each incorrect program was examined after the experiment to determine the basis for its rejection. Analysis of the findings is reported here in terms of omitted or incorrect conditional transfer-of-control commands and non-conditional commands (counter/card movement commands and "go to" commands); these results are shown as entries (c) and (d) respectively in Table 2. The association of errors with "or" and with "mixed" problems is quite pronounced with these measures. While the statistical significance of the differential frequency of occurrence of these three program measures in the problem conditions is not demonstrated, the pattern of findings is pronounced and consistent enough to suggest generalized effects of both problem variables on error characteristics of subjects' programs.*

Incorrect conditional commands constituted the largest error class (33/49, 67%) and were of the following four types: (1) repeated address labels in the "yes" and "no" outcome specifications of conditional commands (three instances); (2) correct address labels present in the command branches but reversed from that appropriate for correct problem solution (nine instances); (3) substitution of an incorrect address label for a correct one in one branch of a command (eight instances); and (4) one incorrect address-label substitution compounded with a reversal, in the same command, of the position of the correct address label (13 instances).

The three repetition errors and possibly the nine reversal errors might be attributed to inattention or carelessness on the part of subjects. However, the remaining two kinds of errors are not easily attributable to such causes in view of the fact that these errors (sometimes accompanied by reversal errors) most often led to programming solutions where "and" sorting concepts were provided for "or" problems. It seems much more likely that these errors reflect erroneous planning as subjects organized their programs.

*As a matter of interest chi-square tests were computed for the measures in Table 2 for both problem variables under the hypothesis that there should be an equal distribution of measures within levels. Significance levels of at least 0.05 were reached for all measures for both variables except for the measure of number of initially incorrect programs ($P < 0.10$).

SORTING CONCEPT OF INCORRECT PROGRAMS

Incorrect programs were also analyzed from the point of view of the sorting concept that was programmed, in relation to the problem specified. Only the 25 programs containing conditional errors were so examined (the remaining seven out of 32 incorrect programs contained only non-conditional errors and could not thereby alter the sorting concept). Twenty-three of these 25 programs involved incorrect sorting concepts for "or" problems, and 21 of these were associated with negative expression of at least one letter test. Sixteen of these 21 programs (nine in the mixed condition) resulted in providing an "and" instead of an "or" sorting concept (five resulted in a sorting concept based on only one letter test). Furthermore, all 16 of these programs had a conditional error located in the first letter test (seven instances did not involve an error in the second test). Lastly, 10 of these 16 programs resulting in an "and" solution for an "or" problem were correct in terms of the form of test-expression condition specified by the problem (i.e. if the problem required a solution for "A or not-B", the solution "A and not-B" was programmed).

Thus, the problem combination of "or" concepts with at least one negative test expression is not only highly associated with program errors, but also the effect of the errors is such as to lead to primarily "and" solutions via incorrect first conditional tests, without change in the test expression condition as stated in the problem.

PROGRAM STRUCTURE

The previous analyses have established that "or" problems were programmed less efficiently than "and" problems, and the combination of "or" problems with negative test expression was associated with higher frequencies of error measures. The present discussion of the structure of subjects' programs, in terms of the sequence of commands and the branching pattern within conditional commands, provides a basis for suggesting an interpretation of these findings.

The program structure analyses were limited to the letter test, counter/card movement, and unconditional transfer commands, since differences with respect to the remaining commands are of little significance. The following coding system was used to characterize the command sequences. The letters "A" and "B" refer to the two conditional commands required in all problems, which commands test for the presence (or absence) of a particular letter in a specific letter position; the letter "G" refers to the unconditional "go to" command. The letter "P", standing for "positive outcome", refers to the card movement and counter-incrementation commands appropriate when the

logical expression of the two tests was true (i.e. "A and B" or "A or B"). The letter "N", standing for "negative outcome", refers to the card-movement command appropriate when the logical expression was false. Using this symbology, the command sequence of the example program given in Fig. 2 is: "A, B, P, G, N, G" (where "P" refers to commands 8 and 9, and "N" refers to command 11).

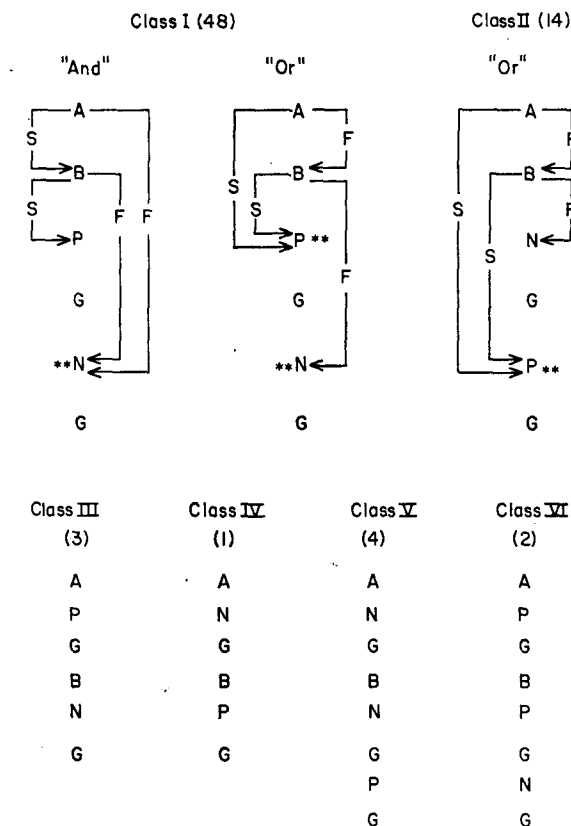


FIG. 3. Classification and frequency (in parentheses) of initial program structures (cf. text for explanation).

A total of six different classes of command sequences were found for the 72 initial programs. These program structures are represented in Fig. 3 in terms of the letter codes introduced above; details of the branching pattern of conditional commands for class I and II structures are given in flow-diagram representation and are discussed below; the frequency of occurrence of each class is given in parentheses.

Whereas the class I and II structures are equally appropriate for programming either "and" or "or" problems, the other structures are applicable primarily only to one problem type due to the separation of conditional tests (class III and VI structures are appropriate for "or" problems; class IV and V structures are appropriate for "and" problems). A further characteristic of the class V and VI structures is the redundant repetition of a command sequence. Only one of the six such redundant structure programs was incorrect, whereas all four of the class III and IV structure programs were incorrect. Given separation of conditional tests, it would appear that the redundancy facilitated specification of the transfer-of-control within these repetitious programs.

The type of program structure generated appears to be associated with the problem conditions. Almost all of the "and" programs were of class I structure (32/36, 89%), while there was much more diversity for "or" programs: 50% were of class I (18/36), 33% were of class II (12/36), and the remainder were more or less equally distributed among the remaining four classes.

There are also associations between choice of class I or II program structures in particular problem conditions and frequencies of error measures. While low frequencies of error measures were associated with class I structures in the "and" problems, this was not the case for "or" problems. Here, the error rates for class I structures were higher than for "and" problems and also higher than for class II structures within the same "or" conditions. For example, 23 of the 25 rejected programs involving conditional errors occurred for "or" problems, and 16 had a class I structure (79%) while only four had a class II structure (17%). Also, for the 30 (out of 33) conditional errors in the "or" problems, 21 occurred with class I structure programs (70%) while only four were associated with class II structures (13%). Similarly, nine of the 12 non-conditional "or" errors occurred with class I structures (75%), while only one (8%) occurred with a class II structure. Finally, of the 16 incorrect "or" programs which resulted in an "and" sorting concept, 13 involved a class I structure (81%) and only one was a class II structure program (6%).

Such findings strongly suggest that while the class I structure was appropriate for "and" problems, its use for "or" problems involved greater planning or organizational difficulties. On the other hand, the class II structure appears to be the less difficult and more appropriate structure for the "or" problems. This suggestion is verified by certain characteristics of the structures observed in the flow-diagram representations given in Fig. 3. It should be noted that the transfer-of-control paths in the diagrams refer to

target-detection outcomes—success (S) or failure (F)—rather than to “yes” and “no” outcomes. This representation renders the flow diagrams independent of the form of test expression. The asterisks beside certain letter codes indicate the presence of identifying address labels.

For the class I “and” structure, the first characteristic of note is the symmetry of the branching paths for the two conditional commands: following successful target detection, transfer is made to the next adjacent command, and, following failure, transfer is made to the negative-outcome command. This symmetry could facilitate planning in “and” problems by permitting transfer of the reasoning underlying specification of the branch pattern for the first conditional command directly to the second without need for re-analysis. In the class I “or” structure this potentially facilitating symmetry is not present.

A second characteristic of the class I “and” structure is that only the negative-outcome command requires an address label, whereas an additional label is required for the positive-outcome commands in the comparable “or” structure. The necessity to select and specify an extra address label (both in the branch of a test and as an identifier of the first “P” command) could interfere with or generally over-burden the subject’s organizational planning if the class I structure is chosen for “or” problems.

A third, less interpretable, difference between the two class I structures is that the path to the positive-outcome commands, presumably of more concern to subjects as a result of problem-statement emphasis, is singular and linear for the “and” structure (i.e. “A–B–P”) whereas in the “or” structure there are two such paths, one of which involves a forward “jump” transfer of control (i.e. “A–B–P” given failure of test A but success of test B, and “A–P”, given success of test A). For whatever reasons, it may be that a successive adjacent sequence of actions to the emphasized positive-outcome actions, via a singular path, may place lower planning demands on subjects than multiple paths involving “jumps”.

For the above reasons, the “or” class I structure appears to require more information processing than the comparable “and” structure. On the other hand examination of the flow diagram of the class II “or” structure indicates that it has the same potentially facilitating characteristics of transfer symmetry and minimal address labels that characterize the class I “and” structure and, in these terms, is more appropriate for the “or” problems. The shift of structure frequencies, from almost unanimous choice of class I for “and” problems to a 33% selection of a class I structure for “or” problems may be related to some subjects’ apprehension or appreciation of the alleged greater complexity of class I structures for “or” problems.

While such characteristics may help to explain choice of particular structures, they do not explain why most of the incorrect class I "or" structures resulted in "and" programs, most of these (10 out of 16) preserving the test expression condition stated in the problem. A general hypothesis, transferred from concept-learning research, that subjects have a general conceptual difficulty with disjunctive as opposed to conjunctive concepts (e.g. Bourne, 1966) might predict errors for "or" programs. However, it is not clear how such a general difficulty hypothesis would predict that "and" sorting concepts would result from errors in "or" programs. Further, such a general hypothesis would predict difficulties for all three "or" conditions, not just those involving a negative test expression. The errors made in the both-affirmative "or" condition were not only fewer than in the other two expression conditions, but might be attributable to carelessness, inattention, etc. (this is certainly true for the non-conditional errors and also for the repetition-type conditional errors which occurred). Finally, subjects' comments are not consistent with such an interpretation, as they indicated understanding "or" concepts well enough but expressed having problems in translating such concepts into a sequential procedure.

A reasonable alternative to a vague conceptual-difficulty hypothesis is one which is specific to the particular task of procedure specification. One factor of possible importance is the fact that subjects had difficulty in detecting errors when "and" solutions were provided for "or" problems. The overall average number of resubmitted programs for the 17 instances of initially incorrect programs was 1.88 (32/17). However, the average number of resubmissions for the five cases in which the initially incorrect program provided an "and" solution for an "or" problem was 3.20 (16/5).

In post-experimental informal questioning of subjects concerning their procedures used to detect program errors ("debugging"), none indicated use of a testing procedure in which the action of the program on sample inputs was examined (as used by professional programmers). Those few who verbalized their procedures indicated that the branching patterns of conditional commands were examined to determine if they "looked right". Presumably, other subjects followed the same passive analytical procedure in their debugging activities. Given the limited debugging experience of subjects, the absence of active tracing of program control flow, and the fact that transfer-of-control is indicated symbolically in these programs (in contrast to the directly perceivable flow paths in flow diagrams; cf. Fig. 3), it is not surprising that subjects were not exceptionally efficient in program debugging (cf. following section).

Generally, poor debugging procedures, however, would not explain the

greater number of resubmissions required for erroneous "and" solutions to "or" problems relative to the overall rate. The occurrence of such errors and the difficulty subjects had in detecting them may be explainable in terms of a hypothesis which presumes influences on behavior from pre-existing experiences with procedures. It is postulated that the bulk of natural experience with procedures is in terms of a sequence of tests in each of which there is some assessment of the state of a relevant variable, followed immediately by actions appropriate to the outcome of the test. In particular, it is hypothesized that most procedures emphasize the actions to be taken following successful detection of some target, and de-emphasize or perhaps not even specify the so-called negative-outcome actions. A further postulated characteristic of natural experience with procedures is that continuation of the action sequence most often depends upon successful target detection at every step; failing any one test further action is discontinued (e.g. the medical deduction, income averaging, etc., procedures of federal income tax forms). While disjunctive concepts may appear, it is further argued that they are typically treated as a single conditional test (e.g. "if any or all of conditions A, B, or C, then do step x") rather than as a sequence of individual tests—as is required when using the present programming language (e.g. "if not-A then test B, if not-B then test C", etc.)

In the present task it appears reasonable that the format of the problem statements may have established the following expectations, consistent with the postulated prior procedure experience: (1) the positive-outcome commands would be perceived as the desired, successful, emphasized, etc., actions, since the card-movement command of the "P" commands was the first-mentioned action in the problem; (2) the statement next of the two targets to be detected could create an expectation that both tests were pre-requisites for specification of the goal actions, as established in (1). The latter expectation is appropriate without re-examination for "and" problems but is inappropriate for "or" problems. To understand "or" problems correctly subjects would have to attend to and understand the qualifying phrases in the problem statement which identified the "or" concept.

The present interpretation is that the negatively-expressed tests were the attentional focus of most subjects having difficulty with "or" problems, resulting from the "awkwardness"—in terms of pre-existing habits—of expressing a test in negation. It is supposed that it is more "natural" to interpret the "yes" response as an indicator of successful detection of a specified target than it is to so interpret the "no" response. In the present programming language there was no provision of operators to allow

testing for the absence of letter values directly (e.g. is first letter "*not-A*") so that the "yes" outcome could indicate target detection.

Given that subjects were attending to the more "local" problem of specifying tests in negation, attention would be diverted from the "global" organizational planning of the transfer structure appropriate for "or" problems. In the absence of the necessary attention, pre-existing tendencies to assume that target detection was necessary for continuation, and prior tendencies associating target detection with positive-outcome actions (and failure with negative-outcome), would dominate subjects' specification of branching patterns for conditional commands. Such patterns would, unintentionally, be appropriate for "and" problems, and, further, they would be difficult to detect in view of their compatibility with prior procedure experience. Such a hypothesis can account for not only the "and" nature of incorrect solutions provided for "or" problems, but also for the absence of such effects in the "or" condition in which both tests were expressed affirmatively; here the requirements of specifying the "or" structure could be attended to without competition of the more demanding problem of expressing negative tests in a manner incompatible with previous experience.

Aside from the fact that the above *ad hoc* interpretation is consistent with the performance findings, the hypothesis has utility for suggesting a direction of future research in procedure specification. Whether the goal is to generate "natural" programming languages highly compatible with pre-existing behavior modes, or whether the interest is in the efficiency characteristics of procedure specification as a problem-solving activity, it would seem important to investigate normative procedure specification activities in "natural" situations, e.g. as expressed in natural English rather than in a restrictive unfamiliar programming language.

The appropriate larger context of these findings concerning the relationship of program structure and performance is the general domain concerning subject's representations of problems and problem information (e.g. "problem space", cf. Simon & Newell, 1972). The present data contribute to the increasingly detailed evidence that subjects' problem solutions are closely related to their choice of such representations (e.g. Schwartz, 1973).

PROGRAM DEBUGGING

Already mentioned is the inference from subjects' comments suggesting they employed a passive analytical procedure for debugging their programs in contrast to an active testing procedure. Quantitative data were also obtained concerning incorrect programs with respect to the number of

modifications made to the programs, the number of times programs were displayed, and the amount of time spent in debugging.

With respect to the last measure, for each of the 17 incorrect programs, a ratio was computed of the time to program the initially submitted program to the total time required to obtain a finally correct program. These normalized data were averaged to obtain a debugging-time factor. Overall, given that a program required one unit of time to program initially, if it was incorrect, 1.58 units of time were required to detect and correct the errors—half again as much time as the whole initial programming activity. In contrast to previous measures, there was no association of this factor with the problem conditions (averages for levels of the two problem variables were within ± 0.10 of the overall debugging-time factor).

The total number of program displays and modifications which occurred during the debugging of the 17 incorrect programs was 98 and 45, respectively. These were distributed among the problem conditions in a manner similar to (and not independent of) the error measures: approximately 75% of both measures occurred for “or” problems, with 10, 55, and 35% occurrences for the both-affirmative, mixed, and both-negative conditions, respectively.

The overall rate per program of program modifications and displays for the 17 incorrect programs was 5.77 and 2.65, respectively. The corresponding rates for the initial 72 programs were 1.72 and 1.86 modifications and displays respectively.

If there is a moral to be drawn from these findings, it is that the exorbitant time, modification, and display requirements of debugging conceptual errors, as contrasted to initial programming, warrants consideration of techniques for the prevention and/or detection of such errors during the initial programming process rather than at a later point.

TESTING RESULTS

Starting with the third subject, after completion and acceptance of a correct program subjects were asked in a pencil and paper test to pick from a list of eight the smallest number of names that would sufficiently test whether their program sorted name cards as required by the problem. Logically, only four names are required, one to test each of the four possible outcomes of the combinations of the two conditional letter tests.

The subjects' selections were evaluated in terms of three measures: total number of names selected (four necessary), number of outcomes tested (out of four possible), and number of names which redundantly tested the same outcome tested by another chosen name.

Over all problems, subjects selected an average of 3.25 (81%) of the necessary four. An average of 2.91 (73%) of the four outcomes was tested, and of the names selected 0.33 (10%) were, on the average, redundant. The distributions of the measures in the problem conditions were roughly of the same magnitude. These results indicate a fairly high level of testing efficiency.

Conclusions

The key points of the present study are as follows:

(1) The feasibility of using naive subjects to investigate programming processes and problems has been demonstrated.

(2) Programming, or procedure specification, has been shown to be influenced by the logical characteristics of the problems interpretable in terms specific to the task.

(3) The transfer-of-control structure of programs appears to be an important determinant of programming efficiency and ultimate program correctness.

(4) An influence of prior experience with procedures on the production of program errors is tentatively indicated, suggesting the utility of research investigating the nature of "natural" procedure specification behavior.

This research was supported in part by a contract from Engineering Psychology Programs, Office of Naval Research.

Appreciation is expressed to Steve Boies, John Gould, and Gerald Weinberg for their helpful comments on an early draft of this paper.

References

- BOIES, S. J. & GOULD, J. D. (1971). User behavior in an interactive computing system. Abstract in the *Proceedings of the Fifth Annual Princeton Conference on Information Sciences and Systems*, 122.
- BOURNE, L. E., JR. (1966). *Human Conceptual Behavior*. Boston: Allyn & Bacon.
- BROPHY, H. F. (1970). Improving programmer performance. *Australian Computer J.*, 2, 66.
- DUNCKER, K. (1945). On problem solving. *Psychological Monographs*, 58 (Whole No. 270).
- GOULD, J. D., DOHERTY, W. J. & BOIES, S. J. (1971). Bibliography of behavioral aspects of on-line computer programming. *IBM Research Report*, RC-3513.
- HEWITT, J. (1972). Programming: challenge for the '70s (comments in magazine article.) *Think*, April. Pp. 40-45.
- MAIER, N. R. F. (1970). *Problem Solving and Creativity in Individuals and Groups*. Belmont, Calif.: Brooks-Cole.

- MAYER, D. B. & STALNAKER, A. U. (1967). Computer personnel research—issue and progress in the 60s. In *Proceedings of the Fifth Annual Computer Personnel Research Conference*. Ed. R. Blechen. Pp. 6–41. Association for Computing Machinery publication.
- MILLER, R. B. (1968). An analysis of human problem solving tasks. *IBM Technical Report* no. AR-0934-00.
- NEWELL, A. & SIMON, H. A. (1972). *Human Problem Solving*. Englewood Cliffs, N.J.: Prentice-Hall.
- PAKIN, S. (1968). *APL/360 Reference Manual*. Chicago: Science Research Associates, Inc.
- SACKMAN, H. (1968). Man-Computer communication: experimental investigation of user effectiveness. In *Proceedings of the Sixth Annual Computer Personnel Research Conference*. Ed. R. Blechen. Pp. 93–105. Association for Computing Machinery publication.
- SACKMAN, H. (1970). *Man-Computer Problem Solving*. New York: Auerbach Publishers, Inc.
- SAMMETT, J. E. (1969). *Programming Languages: History and Fundamentals*. Englewood Cliffs: Prentice-Hall.
- SCHWARTZ, S. H. (1973). Representation in routing problems. Paper presented at the *Midwestern Psychological Association Meeting*, May 10–12.
- WASON, P. C. (1959). The processing of positive and negative information. *Quarterly Journal of Experimental Psychology*, **11**, 92.
- WEINBERG, G. M. (1971). *The Psychology of Computer Programming*. New York: Van Nostrand Reinhold.